Django + Waitress + Nginx + uWSGI on Windows – A to Z Complete Guide

1. What Is Django + Waitress + Nginx?

- ****Django:**** Python web framework used to build websites and APIs.

- ****Waitress:**** WSGI application server that runs Django.

- ****Nginx:**** Web server and reverse proxy that forwards requests to Waitress and serves static files.

- ****Database:**** Stores application data.

Request Flow

```text

Browser

   |

   v

Nginx (Port 80)

   |

   v

Waitress (127.0.0.1:8000)

   |

   v

Django Application

   |

   v

Database

```

Nginx serves static files directly and forwards application requests to Waitress.

---

2. Install Required Software

Install:

1. Python

2. Django

3. Waitress

4. Nginx for Windows

Official resources:

- Python: https://www.python.org/downloads/windows/

- Nginx: https://nginx.org/en/download.html

- Django: https://docs.djangoproject.com/en/stable/

- Waitress: https://docs.pylonsproject.org/projects/waitress/en/stable/

Verify Python:

```bat
python --version

python -m pip --version
```

---

3. Create a Django Project

```bat
```

```
mkdir C:\projects

cd C:\projects

python -m venv .venv

.venv\Scripts\activate

python -m pip install --upgrade pip

pip install django waitress

django-admin startproject myproject

cd myproject
```

Project structure:

```text
C:\projects\\
├── .venv\\
└── myproject\\
    ├── manage.py
    └── myproject\\
        ├── __init__.py
        ├── settings.py
        ├── urls.py
        ├── asgi.py
        └── wsgi.py
```

Run initial migrations:

```bat
python manage.py migrate
```

Test Django locally:

```bat
python manage.py runserver
```

Open:

```text
http://127.0.0.1:8000/
```

Stop the development server with `Ctrl + C`.

---

4. Configure Django Settings

Open:

```text
C:\projects\myproject\myproject\settings.py
```

Configure these settings while preserving your existing applications, middleware, database configuration, and other project settings.

```python
import os
```

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

DEBUG = os.environ.get(

    "DJANGO_DEBUG", "False"

).lower() == "true"

SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY", "")

ALLOWED_HOSTS = [

    "localhost",

    "127.0.0.1",

]

STATIC_URL = "/static/"

STATIC_ROOT = BASE_DIR / "staticfiles"

MEDIA_URL = "/media/"

MEDIA_ROOT = BASE_DIR / "media"
```

Important Settings

- `DEBUG = False`: Disables Django's debug mode.

- `SECRET_KEY`: Should be stored securely outside source code.

- `ALLOWED_HOSTS`: Lists the permitted hostnames.

- `STATIC_ROOT`: Destination for collected static files.

- `MEDIA_ROOT`: Directory for uploaded files.

Do not replace your existing `settings.py` entirely with this snippet. Merge

````
the settings into your project.

### Set Environment Variables

Open Command Prompt:

```bat

set "DJANGO_DEBUG=False"

set "DJANGO_SECRET_KEY=REPLACE_WITH_A_SECURE_RANDOM_SECRET"

```

These commands apply only to the current terminal session.

Replace the placeholder with a secure, randomly generated secret. Ensure the same environment variables are available to the Waitress process.

### Configure a Domain

For local testing:

```python

ALLOWED_HOSTS = [

    "localhost",

    "127.0.0.1",

]

```

For a public website:

```python

ALLOWED_HOSTS = [

    "example.com",

    "www.example.com",
````
````

```
]
```

Replace the example domains with your actual domain names.

---

5. Configure Static Files

Static files include CSS, JavaScript, logos, and icons.

In `settings.py`:

```python
STATIC_URL = "/static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
```

If your project has a source directory named `static`, you may configure:

```python
STATICFILES_DIRS = [
    BASE_DIR / "static",
]
```

Only add this setting if the directory exists.

Collect static files:

```bat
python manage.py collectstatic --noinput
```

````
Example structure:

```text

myproject/

├── manage.py

├── myproject/

├── static/

│   └── css/

│       └── style.css

└── staticfiles/

    └── admin/

```

Nginx will serve files from `staticfiles`.

---

## 6. Configure Media Files

Media files include uploaded images and documents.

In `settings.py`:

```python

MEDIA_URL = "/media/"

MEDIA_ROOT = BASE_DIR / "media"

```

Example:

```text
````
````

```
myproject/
└── media/
    ├── profile_images/
    └── documents/
```

Create the media directory if necessary.

Do not expose private documents through public media URLs. Use appropriate authorization and protected storage for sensitive uploads.

---

7. Configure WSGI

File:

```text
myproject/wsgi.py
```

Typical Django WSGI configuration:

```python
import os
from django.core.wsgi import get_wsgi_application
os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "myproject.settings",
)
application = get_wsgi_application()
```

```

Here, `myproject` is the inner Python package name.

Waitress will use:

```text
myproject.wsgi:application
```

---

8. Install and Run Waitress

Install Waitress:

```bat
cd C:\projects\myproject

C:\projects\.venv\Scripts\activate

pip install waitress
```

Run the Django application:

```bat
waitress-serve --host=127.0.0.1 --port=8000 myproject.wsgi:application
```

Alternatively, use the executable directly:

```bat
C:\projects\.venv\Scripts\waitress-serve.exe --host=127.0.0.1 --port=8000
myproject.wsgi:application
```
```

```

Test:

```text

http://127.0.0.1:8000/

```

Why Use 127.0.0.1?

Binding to `127.0.0.1` makes Waitress listen on the local loopback interface.

Nginx can communicate with Waitress locally without exposing port 8000 directly to the network.

Keep Waitress running while testing Nginx.

---

9. Install Nginx on Windows

1. Download Nginx from https://nginx.org/en/download.html

2. Extract the ZIP archive.

3. Place the extracted files in `C:\nginx`.

4. Verify that `C:\nginx\nginx.exe` exists.

Expected structure:

```text

C:\nginx\\

├── nginx.exe

├── conf\\

|   ├── nginx.conf

|   └── mime.types
```
```

````
├── logs\\
└── html\\
```

Configuration file:

```text
C:\nginx\conf\nginx.conf
```

---

## 10. Complete Copy-Paste-Ready nginx.conf

Assumptions:

- Django project: `C:\projects\myproject`
- Static directory: `C:\projects\myproject\staticfiles`
- Media directory: `C:\projects\myproject\media`
- Waitress: `127.0.0.1:8000`
- Nginx HTTP port: `80`

Replace the contents of `C:\nginx\conf\nginx.conf` with:

```nginx
worker_processes  1;
error_log  logs/error.log warn;
events {
    worker_connections  1024;
}
````

```
http {

    include       mime.types;

    default_type  application/octet-stream;

    access_log    logs/access.log;

    sendfile      on;

    keepalive_timeout  65;

    client_max_body_size 20M;

    upstream django_waitress {

        server 127.0.0.1:8000;

        keepalive 16;

    }

    server {

        listen 80;

        server_name localhost 127.0.0.1;

        # Django static files

        location /static/ {

            alias C:/projects/myproject/staticfiles/;

            expires 30d;

            add_header Cache-Control "public";

        }

        # Uploaded media files

        location /media/ {
```

```
            alias C:/projects/myproject/media/;
            expires 7d;
        }
        # Forward application requests to Waitress
        location / {
            proxy_pass http://django_waitress;
            proxy_http_version 1.1;
            proxy_set_header Connection "";
            proxy_set_header Host $http_host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_connect_timeout 60s;
            proxy_send_timeout 60s;
            proxy_read_timeout 60s;
        }
    }
}
```

How This Configuration Works

Static Files

```nginx
```

```
location /static/ {

    alias C:/projects/myproject/staticfiles/;

}
```

A request for:

```text
/static/css/style.css
```

maps to:

```text
C:/projects/myproject/staticfiles/css/style.css
```

Media Files

```nginx
location /media/ {

    alias C:/projects/myproject/media/;

}
```

Nginx serves files from the configured media directory. Do not use this public location for private uploads.

Reverse Proxy

```nginx
location / {
```

````
    proxy_pass http://django_waitress;
}
```

Application requests are forwarded to Waitress.

#### Forwarded Headers

```nginx
proxy_set_header Host $http_host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

These headers pass request information to the upstream application.

The configuration above supports HTTP. HTTPS requires additional TLS certificate and server configuration.

---

## 11. Run Django Deployment Checks

From the project directory:

```bat
cd C:\projects\myproject
C:\projects\.venv\Scripts\activate
python manage.py check
python manage.py check --deploy
````

```
python manage.py migrate

python manage.py collectstatic --noinput
```

Read any warnings produced by `check --deploy` and address them according to your deployment setup.

Create an admin user:

```bat
python manage.py createsuperuser
```

Follow the prompts to configure the administrator account.

---

12. Test Nginx Configuration

Open Command Prompt:

```bat
cd C:\nginx

nginx.exe -t
```

If the configuration test succeeds, start Nginx:

```bat
start nginx
```

Open:

```text
```

````
http://127.0.0.1/
```

Direct Waitress test:

```text
http://127.0.0.1:8000/
```

Both URLs should work during testing.

---

## 13. Important Nginx Commands

Run these commands from `C:\nginx`.

### Test Configuration

```bat
nginx.exe -t
```

### Start Nginx

```bat
start nginx
```

### Reload Configuration

```bat
nginx.exe -s reload
```
````
````

````
### Gracefully Stop Nginx

```bat

nginx.exe -s quit

```

### Stop Nginx Immediately

```bat

nginx.exe -s stop

```

Always test the configuration before reloading:

```bat

cd C:\nginx

nginx.exe -t

nginx.exe -s reload

```

Restart Waitress separately when its application code, settings, or
environment changes require a restart.

---

## 14. Windows Firewall and Networking

For local testing, use:

```text

http://127.0.0.1/

```

For public access, you may need:
````

1. A public IP address or domain.

2. Correct DNS records.

3. Windows Firewall inbound rules.

4. Cloud firewall or router port forwarding, if applicable.

5. HTTPS configuration and a valid TLS certificate.

Typical public ports:

- `80`: HTTP

- `443`: HTTPS

Do not expose Waitress port `8000` publicly. Keep Waitress bound to `127.0.0.1`.

Opening port 443 does not enable HTTPS by itself; the TLS configuration and certificate must also be in place.

---

15. Troubleshooting Common Errors

502 Bad Gateway

Possible causes:

- Waitress is not running.

- The upstream port is incorrect.

- Django failed to start.

- Nginx cannot connect to `127.0.0.1:8000`.

Check Waitress and Nginx logs.

Static Files Return 404

Run:

```bat

python manage.py collectstatic --noinput

```

Verify that `STATIC_ROOT` matches the Nginx `alias` directory.

Invalid HTTP_HOST / Bad Request

Check `ALLOWED_HOSTS` in Django settings and ensure it includes the hostname used by the browser.

403 Forbidden

Check the configured directory paths, Nginx file permissions, and error logs.

Port 80 Already in Use

Another application may already be listening on port 80. Identify the conflict and resolve it before starting Nginx.

Django Import Error

Verify:

- The virtual environment is correct.

- All required packages are installed.

- The package name in `myproject.wsgi:application` is correct.

- `DJANGO_SETTINGS_MODULE` points to the correct settings module.

View Nginx Logs

```bat

type C:\nginx\logs\error.log

type C:\nginx\logs\access.log

```

```
Also check the terminal output or service logs for Waitress.

---

## 16. Keep Waitress Running

A process started from Command Prompt may stop when the terminal closes.

For a persistent Windows deployment, consider a Windows service wrapper such as NSSM:

https://nssm.cc/

Configure the service with:

- ****Application:**** `C:\projects\.venv\Scripts\waitress-serve.exe`

- ****Arguments:**** `--host=127.0.0.1 --port=8000 myproject.wsgi:application`

- ****Working directory:**** `C:\projects\myproject`

- ****Environment:**** Production environment variables

- ****Permissions:**** Only the permissions required by the application

Configure Nginx startup separately.

After configuration, reboot the machine and verify that both Nginx and Waitress start correctly.

---

## 17. Production Security Checklist

Before deploying publicly:

- Set `DEBUG=False`.

- Keep `SECRET_KEY` outside source control.

- Configure `ALLOWED_HOSTS` correctly.
```

````
- Configure HTTPS and a valid certificate.

- Review CSRF settings and secure cookie settings.

- Verify `SECURE_PROXY_SSL_HEADER` only when the proxy setup is trusted.

- Secure database credentials.

- Configure database and media backups.

- Do not expose private uploads through public media URLs.

- Keep Waitress bound to `127.0.0.1`.

- Restrict unnecessary inbound ports.

- Configure log retention and monitoring.

- Test backup restoration and rollback procedures.

- Run `python manage.py check --deploy`.

Do not enable HSTS or force HTTPS until HTTPS is correctly configured and
verified.

---

## 18. Redeploy After Code Changes

A basic deployment workflow:

```bat

cd C:\projects\myproject

C:\projects\.venv\Scripts\activate

git pull

pip install -r requirements.txt

python manage.py check --deploy

python manage.py migrate
````
````

```
python manage.py collectstatic --noinput
```

Then:

1. Restart the Waitress service.

2. Test the Nginx configuration if it changed.

3. Reload Nginx when appropriate.

4. Test the website, authentication, APIs, static files, and uploads.

5. Check application and Nginx logs.

Before deploying, review database migration compatibility and maintain a backup and rollback plan.

---

19. Waitress vs Gunicorn vs uWSGI

Server	Typical use
Waitress	WSGI applications on Windows and other supported platforms
Gunicorn	Common WSGI deployment on Linux/Unix
uWSGI	Commonly used in Linux/Unix application deployments
Uvicorn	ASGI applications such as FastAPI and Django ASGI

Waitress is a practical WSGI option for Windows-hosted Django projects.

If your application requires ASGI features, such as certain asynchronous workloads or Django Channels, verify that your selected application server supports your requirements.

````
## 20. Recommended Linux Production Architecture

For many public, high-traffic deployments, Linux is a common choice because of its process management ecosystem and the limitations of the Windows Nginx build.

Typical architecture:

```text

Linux Server

    |

    v

Nginx

    |

    v

Gunicorn / Uvicorn

    |

    v

Django

    |

    v

PostgreSQL

```

Choose the deployment architecture based on your hosting environment and application requirements.

---

## 21. Final Quick-Start Commands
````

````
Assumptions:

- Project: `C:\projects\myproject`

- Virtual environment: `C:\projects\.venv`

- Nginx: `C:\nginx`

- WSGI package: `myproject`

### Terminal 1: Django Checks and Static Files

```bat

cd C:\projects\myproject

C:\projects\.venv\Scripts\activate

python manage.py check --deploy

python manage.py migrate

python manage.py collectstatic --noinput

```

Ensure the production environment variables and database configuration are
correct before running these commands.

### Terminal 2: Start Waitress

```bat

cd C:\projects\myproject

C:\projects\.venv\Scripts\waitress-serve.exe --host=127.0.0.1 --port=8000
myproject.wsgi:application

```

### Terminal 3: Start Nginx

```bat
````
````

```
cd C:\nginx

nginx.exe -t

start nginx
```

Open:

```text
http://127.0.0.1/
```

If everything works, configure the domain, HTTPS, persistent services, firewall, backups, and monitoring before considering the public deployment production-ready.

---

22. Official Documentation

- Django deployment: https://docs.djangoproject.com/en/stable/howto/deployment/

- Django static files: https://docs.djangoproject.com/en/stable/howto/static-files/

- Waitress documentation: https://docs.pylonsproject.org/projects/waitress/en/stable/

- Nginx documentation: https://nginx.org/en/docs/

- NSSM: https://nssm.cc/

****Final note:**** This guide assumes Windows paths and a WSGI-based Django project. Replace example project names, directories, hostnames, and environment variables with your actual configuration.

uWSGI – Complete Notes

````
## 1. What is uWSGI?

****uWSGI**** is an application server that can run Python web applications using the WSGI interface.

It is commonly used to deploy Django applications in production environments, often together with Nginx.

### Example

During development, we run Django using:

```bash

python manage.py runserver

```

For production deployment, we can use uWSGI to run the Django application.

### Main Components

- ****Django:**** Python web framework.

- ****WSGI:**** Standard interface between a Python web application and a WSGI server.

- ****uWSGI:**** Server that can run WSGI applications.

- ****Nginx:**** Web server and reverse proxy.

- ****Database:**** Stores application data.

### Request Flow

```text

Client / Browser

       |

       v

     Nginx
````
````

```
       |
       v
     uWSGI
       |
       v
     Django
       |
       v
    Database
```

---

2. What is WSGI?

****WSGI**** stands for Web Server Gateway Interface.

It is a standard interface between Python web applications and web servers.

For example, Django provides a WSGI application through its `wsgi.py` file.

```python
import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault(

    "DJANGO_SETTINGS_MODULE",

    "myproject.settings",

)
```

````
application = get_wsgi_application()

```

Here:

- `DJANGO_SETTINGS_MODULE` specifies the Django settings module.

- `get_wsgi_application()` creates the WSGI application.

- `application` is the callable used by a WSGI server.

****Important:**** WSGI is a standard interface, while uWSGI is a server that can implement and use that interface.

---

## 3. What is pip install uwsgi?

The command below installs the uWSGI Python package using pip.

```bash

pip install uwsgi

```

A recommended alternative is:

```bash

python -m pip install uwsgi

```

Using `python -m pip` helps ensure that pip belongs to the selected Python interpreter.

### Check Python Version

```bash

python --version
````

```

### Check pip Version

```bash
python -m pip --version
```

---

4. Create a Virtual Environment

A virtual environment keeps project dependencies isolated.

Create Virtual Environment

```bash
python -m venv venv
```

Activate on Windows CMD

```bat
venv\Scripts\activate
```

Activate on Windows PowerShell

```powershell
.\venv\Scripts\Activate.ps1
```

Activate on Linux or macOS

```bash
```
```

```
source venv/bin/activate
```

After activation, the terminal usually displays `(venv)`.

Upgrade pip

```bash
python -m pip install --upgrade pip
```

Install uWSGI

```bash
python -m pip install uwsgi
```

****Note:**** uWSGI is primarily intended for Unix-like systems. Installation may fail on Windows because it requires native C compilation and Unix-specific functionality. Linux or WSL2 is recommended for learning uWSGI deployment.

---

5. Verify uWSGI Installation

Check Installed Package

```bash
python -m pip show uwsgi
```

Check uWSGI Version

```bash
uwsgi --version
```

```

### Check Executable Location on Linux

```bash

which uwsgi

```

Check Executable Location on Windows

```bat

where uwsgi

```

If the command is not found, verify that the virtual environment is activated and the package is installed in the correct environment.

---

6. Common Installation Errors

Error 1: Failed building wheel for uwsgi

****Cause:**** The system may be missing a C compiler or Python development headers.

On Ubuntu or Debian:

```bash

sudo apt update

sudo apt install build-essential python3-dev

```

Then retry:

```bash

```
python -m pip install uwsgi
```

Error 2: Python.h: No such file or directory

****Cause:**** Python development headers are missing.

Solution on Ubuntu or Debian:

```bash
sudo apt install python3-dev
```

If your system uses a specific Python version, install the matching development package.

Error 3: uwsgi is not recognized

****Cause:**** The executable may not be available on PATH.

Check the installation:

```bash
python -m pip show uwsgi
```

On Linux, try running the executable directly:

```bash
./venv/bin/uwsgi --version
```

Error 4: ModuleNotFoundError: No module named 'django'

****Cause:**** Django is not installed in the Python environment used by uWSGI.

Solution:

```bash
python -m pip install django
```

For an existing project, install the dependencies from its requirements file instead.

---

7. Create a Django Project

Install Django and uWSGI:

```bash
python -m pip install django uwsgi
```

Create a project:

```bash
django-admin startproject myproject
```

Move into the project directory:

```bash
cd myproject
```

Project Structure

```text
```

```
myproject/
├── manage.py
└── myproject/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

Run database migrations:

```bash
python manage.py migrate
```

---

8. Run Django Using uWSGI

Run this command from the directory containing `manage.py`:

```bash
uwsgi --http 127.0.0.1:8000 --module myproject.wsgi:application
```

Open the following address in your browser:

```text
http://127.0.0.1:8000
```

```

### Explanation

- `uwsgi`: Starts the uWSGI server.

- `--http`: Enables an HTTP listener.

- `127.0.0.1:8000`: Specifies the local IP address and port.

- `--module`: Loads the specified Python module.

- `myproject.wsgi:application`: Refers to the `application` callable in `myproject/wsgi.py`.

****Note:**** This command is useful for local testing. Production deployments usually place Nginx in front of uWSGI.

---

## 9. What is uwsgi.ini?

`uwsgi.ini` is a configuration file used to store uWSGI options.

Instead of specifying every option on the command line, we can define them in a file.

### Example: uwsgi.ini

```ini

[uwsgi]

chdir = /path/to/myproject

module = myproject.wsgi:application

http = 127.0.0.1:8000

master = true

processes = 2

threads = 2
```
```

```
vacuum = true

die-on-term = true
```

Replace `/path/to/myproject` with the actual project directory.

Run Using Configuration File

```bash
uwsgi --ini uwsgi.ini
```

Important Options

Option	Description
`chdir`	Sets the application working directory.
`module`	Specifies the Python module and application callable.
`http`	Starts an HTTP listener.
`socket`	Creates a socket endpoint, commonly for Nginx using the uWSGI protocol.
`master`	Enables the master process.
`processes`	Sets the number of worker processes.
`threads`	Sets the number of threads per worker process.
`vacuum`	Cleans up relevant resources created by uWSGI on exit.
`die-on-term`	Supports clean termination when receiving SIGTERM.

****Important:**** The `http` and `socket` options are not interchangeable in every setup. The backend protocol must match the way Nginx connects to uWSGI.

````
---

## 10. uWSGI with Nginx

Nginx is commonly used as a reverse proxy in production deployments.

### Architecture

```text

Browser

   |

   v

Nginx (Port 80 / 443)

   |

   v

uWSGI (Unix Socket)

   |

   v

Django Application

   |

   v

Database

```

### Example: uWSGI Unix Socket Configuration

```ini

[uwsgi]
````
````

```
chdir = /srv/myproject

module = myproject.wsgi:application

master = true

processes = 4

threads = 2

socket = /run/uwsgi/myproject.sock

chmod-socket = 660

vacuum = true

die-on-term = true
```

The project path, socket directory and permissions must be configured for your server.

Example: Nginx Configuration

```nginx
server {

    listen 80;

    server_name example.com;

    location / {

        include uwsgi_params;

        uwsgi_pass unix:/run/uwsgi/myproject.sock;

    }

}
```

Here:

- `listen 80` accepts HTTP traffic on port 80.

- `server_name` specifies the domain.

- `include uwsgi_params` provides parameters used by the uWSGI protocol.

- `uwsgi_pass` forwards requests to the uWSGI Unix socket.

The Nginx worker must have permission to access the socket.

Collect Static Files

```bash

python manage.py collectstatic

```

Configure Nginx to serve the collected static files from the correct directory.

---

11. What is a Master Process?

The uWSGI master process manages worker processes.

Workers handle application requests.

```text

          Master Process

                |

        -----------------

        |       |       |

      Worker  Worker  Worker
```

```
        1       2       3
```

Example

```ini
master = true
processes = 3
```

This configuration enables a master process and three worker processes.

The appropriate worker count depends on CPU resources, memory, workload and application behavior.

---

12. What are Threads?

A thread is an execution unit within a process.

uWSGI can be configured to use multiple threads per worker.

```ini
processes = 4
threads = 2
```

This configures four worker processes with two threads per process.

It provides eight configured worker threads in total, excluding the master process.

****Important:**** More processes or threads do not automatically mean better performance. Benchmark the application and monitor CPU and memory usage before increasing them.

````
---

## 13. Run uWSGI Using systemd

On Linux, systemd can manage the uWSGI service and restart it when required.

Create a service file:

```text

/etc/systemd/system/myproject-uwsgi.service

```

### Example Configuration

```ini

[Unit]

Description=uWSGI for myproject

After=network.target

[Service]

User=www-data

Group=www-data

WorkingDirectory=/srv/myproject

ExecStart=/srv/myproject/venv/bin/uwsgi --ini /srv/myproject/uwsgi.ini

Restart=always

KillSignal=SIGTERM

[Install]

WantedBy=multi-user.target

```
````

Ensure the project directory, virtual environment, socket permissions and service user are correct before starting the service.

Reload systemd

```bash
sudo systemctl daemon-reload
```

Enable and Start Service

```bash
sudo systemctl enable --now myproject-uwsgi
```

Check Service Status

```bash
sudo systemctl status myproject-uwsgi
```

Restart Service

```bash
sudo systemctl restart myproject-uwsgi
```

View Logs

```bash
sudo journalctl -u myproject-uwsgi -f
```

````
## 14. Production Deployment Checklist

Before deploying a Django application with uWSGI:

- Set `DEBUG = False`.

- Configure `ALLOWED_HOSTS`.

- Store secret keys and sensitive credentials securely.

- Configure HTTPS.

- Configure Nginx correctly.

- Set appropriate file and socket permissions.

- Run database migrations.

- Run `collectstatic` when needed.

- Configure logging and monitoring.

- Test the systemd service and restart behavior.

- Use appropriate worker and thread counts.

- Run Django's deployment checks.

Run Django deployment checks with:

```bash

python manage.py check --deploy

```

---

## 15. Quick Revision Cheat Sheet

| Task | Command |

|---|---|
````

```
| Install uWSGI | `python -m pip install uwsgi` |

| Check package | `python -m pip show uwsgi` |

| Check version | `uwsgi --version` |

| Run Django | `uwsgi --http 127.0.0.1:8000 --module
myproject.wsgi:application` |

| Load configuration | `uwsgi --ini uwsgi.ini` |

| Run migrations | `python manage.py migrate` |

| Collect static files | `python manage.py collectstatic` |

| Check systemd service | `sudo systemctl status myproject-uwsgi` |

| View service logs | `sudo journalctl -u myproject-uwsgi -f` |

---

## 16. Important Differences

| Term | Meaning |

|---|---|

| WSGI | Standard interface for Python web applications. |

| uWSGI | Server and application container that supports WSGI. |

| Gunicorn | Alternative Python WSGI server. |

| Nginx | Web server and reverse proxy. |

| ASGI | Interface designed to support asynchronous Python web applications.
|

| `runserver` | Django development server. |

****Final Summary:****

`pip install uwsgi` installs uWSGI into the selected Python environment. A
Django application can then be served through its WSGI callable, typically
```

```
`myproject.wsgi:application`. For production deployment, uWSGI is commonly combined with Nginx, appropriate socket permissions and a service manager such as systemd.

****Official Documentation:****

- uWSGI Installation: https://uwsgi.readthedocs.io/en/latest/Install.html

- uWSGI Quickstart: https://uwsgi.readthedocs.io/en/latest/WSGIquickstart.html

- uWSGI Configuration: https://uwsgi.readthedocs.io/en/latest/Configuration.html

- Django Deployment with uWSGI: https://docs.djangoproject.com/en/stable/howto/deployment/wsgi/uwsgi/

- Django Deployment Checklist: https://docs.djangoproject.com/en/stable/howto/deployment/checklist/
```